**RAJALAKSHMI ENGINEERING COLLEGE**
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

# CS23431 - OPERATING SYSTEM

## EXP 5: Banker's Algorithm Using C

**QUESTION :**

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise,indicate that a deadlock may occur.

## CODE:

```
#include <stdio.h>

int main() {
    int n, m;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    printf("Enter number of resource types: \n");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources: \n");
    for(int j = 0; j < m; j++)
        scanf("%d", &avail[j]);
```

```
    // Need = Max - Allocation
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int finish[n], safeSeq[n], work[m];

    for(int i = 0; i < n; i++)
}
}
```

# Output:

Custom Test — Success

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```